

■ Lab 2 — Resource Block : EC2 & Security Group (GitPod)

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Compris la structure d'un **resource block** Terraform
- Déployé une **instance EC2** et un **Security Group sans règles** sur AWS
- Utilisé le workflow complet : `terraform init` → `validate` → `fmt` → `plan` → `apply` → `destroy`

■ Étape 1 — Reprendre votre environnement GitPod

1. Allez sur ■ <https://gitpod.io/workspaces>
2. Cliquez sur "**Open**" pour relancer votre environnement de formation
3. Ouvrez un terminal : `` Ctrl+ ```

Vérifiez que Terraform est disponible :

```
terraform version
```

Placez-vous dans le dossier du lab :

```
cd labs/lab2-basics
```

■ Étape 2 — Créer les fichiers

Créez les 3 fichiers du lab directement depuis le terminal ou via l'explorateur de fichiers GitPod :

```
touch versions.tf main.tf outputs.tf
```

■ Dans GitPod, vous pouvez éditer les fichiers directement dans l'éditeur VS Code intégré — cliquez sur le fichier dans l'explorateur à gauche.

■ Étape 3 — Bloc ``terraform`` et ``provider``

Ouvrez `versions.tf` dans l'éditeur et collez :

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "eu-west-1"
}
```

■ Étape 4 — Security Group et Instance EC2

Ouvrez `main.tf` et collez :

```
resource "aws_security_group" "lab2_sg" {
  name          = "lab2-security-group"
  description   = "Security group sans regles - Lab 2"

  tags = {
    Name = "lab2-sg"
    Lab  = "lab2"
  }
}

resource "aws_instance" "lab2_instance" {
  ami              = "ami-0694d931cee176e7d"
  instance_type    = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab2_sg.id]

  tags = {
    Name = "lab2-instance"
    Lab  = "lab2"
  }
}
```

■ `aws_security_group.lab2_sg.id` crée une **dépendance implicite** — Terraform sait qu'il doit créer le SG avant l'instance.

■ Étape 5 — Outputs

Ouvrez `outputs.tf` et collez :

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value      = aws_instance.lab2_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value      = aws_instance.lab2_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value      = aws_security_group.lab2_sg.id
}

output "security_group_name" {
  description = "Nom du Security Group"
  value      = aws_security_group.lab2_sg.name
}
```

■ Étape 6 — Workflow Terraform

```
# Initialiser
terraform init

# Valider
terraform validate

# Formater
terraform fmt

# Planifier
terraform plan

# Appliquer
terraform apply
```

Tapez `yes` pour confirmer.

```
# Voir les outputs
terraform output
```

■ Étape 7 — Vérification sur AWS

```
aws ec2 describe-instances \
  --filters "Name=tag:Lab,Values=lab2" \
  --query "Reservations[*].Instances[*].[InstanceId,State.Name,PublicIpAddress]" \
  --output table

aws ec2 describe-security-groups \
  --filters "Name=tag:Lab,Values=lab2" \
  --query "SecurityGroups[*].[GroupId,GroupName]" \
  --output table
```

■ Étape 8 — Nettoyage

```
terraform destroy
```

Tapez **yes** pour confirmer.

■ Validation du Lab

#	Critère	Validé
1	`terraform init` réussi, provider AWS téléchargé	■
2	`terraform validate` retourne "Success"	■
3	`terraform plan` affiche 2 ressources à créer	■
4	`terraform apply` crée l'instance EC2 et le SG	■
5	`terraform output` affiche l'instance ID et le SG	■
6	`terraform destroy` supprime les 2 ressources	■

■ ****Fin du Lab 2 — Première infrastructure déployée avec Terraform !****

Le Lab 3 explore la gestion du ****State file**** : `terraform state list`, `show`, `mv` et `refresh`.